

claude-windows / 577fd4df-ed3f-43d3-854d-d63d732e2e07

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\577fd4df-ed3f-43d3-854d-d63d732e2e07\subagents\workflows\wf_e5e25aeb-1c0\agent-ae14d4e9f567e1aa7.jsonl`
- SHA-256: `cbc6b968746e5e280e4711de08a9b97940784f874a65d3f2cedd091b6875a287`
- Source modified: `2026-06-20T16:36:40+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_e5e25aeb-1c0`
- session_id: `577fd4df-ed3f-43d3-854d-d63d732e2e07`
```

Transcript

1. user (2026-06-20T16:34:09.928Z)

You are reviewing a NEW feature (P3) in a TypeScript SPA: a DETERMINISTIC, dependency-free query compiler that turns plain text ("over 10s", "top 15 longest", "at least 8 shots") into a filter+sort+limit selection over detected badminton rallies. NO LLM, NO network.

Read these files with the Read tool before judging:

```
- CORE parser:      C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts
- its unit tests:   C:/Users/avidu/Projects/khelsutra/web/src/lib/query.test.ts
- QueryBar component: C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx
- App wiring:       C:/Users/avidu/Projects/khelsutra/web/src/App.tsx
- selection hook:   C:/Users/avidu/Projects/khelsutra/web/src/hooks/useRallySelection.ts
- RallyGrid:        C:/Users/avidu/Projects/khelsutra/web/src/components/RallyGrid.tsx
- engine data types: C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts
- the DESIGN CONTRACT: C:/Users/avidu/Projects/khelsutra/docs/PER_RALLY_SELECTION.md (read $2 decisions D5, $3 honest-vs-fabricated fields, $4.3 "Dead-ends / do-NOT", $9 DoD)
```

Report ONLY concrete, defensible findings with exact evidence (an input string ? the wrong parsed output, or a code line). Do NOT invent issues to fill a quota; an empty findings list is a valid, respected answer. Prefer a few high-confidence findings over many speculative ones.

YOUR DIMENSION: robustness, edge cases, and regex hygiene. Probe:

- catastrophic backtracking / ReDoS risk in any RegExp (the module-level global regexes).
- shared module-level global RegExp objects reused across parseQuery() calls ? any lastIndex/state bleed between calls (matchAll vs test vs replace)? Construct a sequence of calls that would break.
- huge numbers, 0, negative-looking input, decimals like "10.5.5", unicode (? ?), commas, extra spaces.
- limit = 0 or huge; NaN; Number() coercion surprises.
- runQuery purity (does it mutate inputs?), sortRallies stability, undefined-field ordering.
- React wiring: the App's display-order vs selection-set separation, stable ordinal map, hook.apply.

Give a concrete failing input/sequence for each.

2. assistant (2026-06-20T16:34:12.516Z)

I'll review the query compiler for robustness, edge cases, and regex hygiene. Let me start by reading the core files.

3. user (2026-06-20T16:34:12.992Z)

```
1      // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
```

```

2      // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable
axes the
3      // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4      // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5      //
6      // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7      // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8      // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9      import type { RallySegment } from "../api/types";
10
11      export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12      export type FilterField = "duration" | "shot_count";
13      /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14      export interface Filter {
15          field: FilterField;
16          op: CompareOp;
17          value: number; // seconds for duration; a raw count for shot_count
18      }
19
20      export type SortKey = "duration" | "start_time" | "shot_count";
21      export interface Sort {
22          key: SortKey;
23          dir: "asc" | "desc";
24      }
25
26      /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27      export type UnavailableKind = "shot-type" | "player" | "score";
28      export interface Unavailable {
29          token: string;
30          kind: UnavailableKind;
31      }
32
33      export interface ParsedQuery {
34          raw: string;
35          /** ANDed together. */
36          filters: Filter[];
37          sort?: Sort;
38          /** "top N" / "first N" / "N longest". */
39          limit?: number;
40          /** Concepts we understood but cannot honour (shot-type / player / score). */
41          unavailable: Unavailable[];
42          /** True when at least one actionable clause (filter | sort | limit) was understood.
*/
43          recognized: boolean;
44      }
45
46      export interface QueryResult {
47          /** ALL rallies in display order (sorted if a sort was given, else input order). */
48          ordered: RallySegment[];
49          /** Ids matching the filters (then limited), in display order. */
50          selectedIds: number[];
51      }
52
53      // ?? grammar fragments ?????????????????????????????????????????????????????????
54      const NUM = "(\\d+(?:\\.\\d+)?)";
55      // Optional trailing unit/noun. "shots?" is listed FIRST so it can't be eaten by the
bare "s"
56      // time-unit alternative; \b stops "m" matching "minutes" etc.
57      const UNIT = "(?:\\s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)\\b)?"
58

```

```

59     interface OpPattern {
60         op: CompareOp;
61         re: RegExp;
62         /** Which capture group holds the number / the unit. */
63         numAt: number;
64         unitAt: number;
65     }
66
67     // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
their
68     // bare counterparts ("more than" / "less than") so the negated phrase is consumed
first.
69     function lead(alt: string, op: CompareOp): OpPattern {
70         return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}`, "g"), numAt: 1, unitAt: 2
};
71     }
72     const LEADING: OpPattern[] = [
73         lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
74         lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),
75         lead("exactly|equal to|equals", "="),
76         lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
77         lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
78     ];
79
80     // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
or the
81     // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
present.
82     interface TrailPattern {
83         op: CompareOp;
84         re: RegExp;
85     }
86     const POST_NOUN = "(?:\\s*(shots?)\\b)?"
87     const TRAILING: TrailPattern[] = [
88         { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater))`${POST_NOUN}`, "g") },
89         { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
(?:less|fewer|shorter|under|below)`${POST_NOUN}`, "g") },
90     ];
91
92     function toSeconds(n: number, unit?: string): number {
93         return unit && /^m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
94     }
95
96     function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
97         const n = Number(numStr);
98         const isShots = !!unit && /^shots?$/i.test(unit);
99         return isShots
100             ? { field: "shot_count", op, value: n }
101             : { field: "duration", op, value: toSeconds(n, unit) };
102     }
103
104     const SORT_PATTERNS: Array<RegExp, Sort> = [
105         [/b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, {
key: "duration", dir: "desc" }],
106         [/b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
"asc" }],
107         [/b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
dir: "desc" }],
108         [/b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
"start_time", dir: "asc" }],
109         [/b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
desc)\b/, { key: "shot_count", dir: "desc" }],
110         [/b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
"shot_count", dir: "asc" }],

```

```

111     ];
112
113     // Order before kind so a token claimed by an earlier kind isn't double-counted.
114     const UNAVAILABLE_PATTERNS: Array<[UnavailableKind, RegExp]> = [
115         ["shot-type", /\b(smash(?:es)?|clears?|drops?|dropshots?|net
?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?)\b/g],
116         ["player", /\b(players?|opponents?|servers?|receivers?)\b/g],
117         ["score", /\b(points?|score|deuce|game point|set point|match point)\b/g],
118     ];
119
120     // ?? public API ?????????????????????????????????????????????????????????
121
122     export function parseQuery(raw: string): ParsedQuery {
123         const normalized = raw
124             .toLowerCase()
125             .replace(/?/g, ">=")
126             .replace(/?/g, "<=")
127             .replace(/,/g, " ")
128             .replace(/\s+/g, " ")
129             .trim();
130
131         const filters: Filter[] = [];
132         let working = normalized;
133
134         // Extract comparator clauses, blanking each match so later passes can't re-read its
135         // number.
136         for (const pat of LEADING) {
137             working = working.replace(pat.re, (full, ...g) => {
138                 filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
139                 return " ".repeat(full.length);
140             });
141         }
142         for (const pat of TRAILING) {
143             working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
144                 filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
145                 return " ".repeat(full.length);
146             });
147         }
148         // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
149         // intent).
150         let sort: Sort | undefined;
151         for (const [re, s] of SORT_PATTERNS) {
152             if (re.test(working)) {
153                 sort = s;
154                 break;
155             }
156         }
157         // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest
158         // N").
159         let limit: number | undefined;
160         let m: RegExpMatchArray | null;
161         if ((m = working.match(/\b(\d+)\s+(longest|shortest)\b/)) || (m =
162         working.match(/\b(longest|shortest)\s+(\d+)\b/))) {
163             const num = /\d/.test(m[1]) ? m[1] : m[2];
164             const word = /\d/.test(m[1]) ? m[2] : m[1];
165             limit = parseInt(num, 10);
166             if (!sort) sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
167         } else if ((m = working.match(/\b(?:top|best)\s+(\d+)\b/))) {
168             limit = parseInt(m[1], 10);
169             if (!sort) sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
170         } else if ((m = working.match(/\b(?:first|earliest)\s+(\d+)\b/))) {
171             limit = parseInt(m[1], 10);
172             if (!sort) sort = { key: "start_time", dir: "asc" };
173         }
174     }

```

```

171     } else if ((m = working.match(/\b(?:last|latest|newest|most recent)\s+(\d+)\b/))) {
172         limit = parseInt(m[1], 10);
173         if (!sort) sort = { key: "start_time", dir: "desc" };
174     }
175
176     // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
177     const unavailable: Unavailable[] = [];
178     const seen = new Set<string>();
179     for (const [kind, re] of UNAVAILABLE_PATTERNS) {
180         for (const mm of normalized.matchAll(re)) {
181             const token = mm[1];
182             if (!seen.has(token)) {
183                 seen.add(token);
184                 unavailable.push({ token, kind });
185             }
186         }
187     }
188
189     return {
190         raw,
191         filters,
192         sort,
193         limit,
194         unavailable,
195         recognized: filters.length > 0 || sort !== null || limit !== null,
196     };
197 }
198
199 function rallyValue(r: RallySegment, key: SortKey): number | undefined {
200     if (key === "duration") return r.duration;
201     if (key === "start_time") return r.start_time;
202     return r.metadata?.shot_count;
203 }
204
205 /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
206 */
207 export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
208     return rallies
209         .map((r, i) => ({ r, i }))
210         .sort((a, b) => {
211             const av = rallyValue(a.r, sort.key);
212             const bv = rallyValue(b.r, sort.key);
213             if (av == null && bv == null) return a.i - b.i;
214             if (av == null) return 1;
215             if (bv == null) return -1;
216             if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
217             return a.i - b.i; // tie ? preserve input order
218         })
219         .map((x) => x.r);
220 }
221
222 function matchesFilter(r: RallySegment, f: Filter): boolean {
223     const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
224     if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
225     switch (f.op) {
226         case ">":
227             return v > f.value;
228         case ">=":
229             return v >= f.value;
230         case "<":
231             return v < f.value;
232         case "<=":
233             return v <= f.value;
234         case "=":
235             return v === f.value;

```

```

235         default:
236             return false;
237     }
238 }
239
240 /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
241     * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
242     * so garbage input can never silently wipe a user's manual curation. The App also
guards on
243     * `q.recognized` before applying, so an empty box is a no-op end to end. */
244 export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
245     if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
246     const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
247     let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
248     if (q.limit !== null) matched = matched.slice(0, q.limit);
249     return { ordered, selectedIds: matched.map((r) => r.id) };
250 }
251
252 // ?? human-readable labels (shared by the QueryBar pill and the tests) ?????????
253 const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
254
255 export function filterLabel(f: Filter): string {
256     if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
257     return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
258 }
259
260 export function sortLabel(s: Sort): string {
261     if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
262     if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
263     return s.dir === "desc" ? "most shots first" : "fewest shots first";
264 }
265
266 /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
15"]). */
267 export function summarize(q: ParsedQuery): string[] {
268     const parts: string[] = [];
269     for (const f of q.filters) parts.push(filterLabel(f));
270     if (q.sort) parts.push(sortLabel(q.sort));
271     if (q.limit !== null) parts.push(`top ${q.limit}`);
272     return parts;
273 }
274

```

4. user (2026-06-20T16:34:13.930Z)

```

1     import { describe, it, expect } from "vitest";
2     import {
3         parseQuery,
4         runQuery,
5         sortRallies,
6         filterLabel,
7         sortLabel,
8         summarize,
9     } from "../query";
10    import type { RallySegment } from "../api/types";
11
12    const mk = (
13        id: number,
14        start_time: number,
15        duration: number,
16        shot_count?: number,

```

```

17 ): RallySegment => ({
18   id,
19   start_time,
20   end_time: start_time + duration,
21   duration,
22   server: null,
23   receiver: null,
24   metadata: shot_count == null ? {} : { shot_count },
25 });
26
27 // A small, varied corpus: ids ordered chronologically, durations & shot counts mixed,
28 // rally 4 deliberately has NO shot_count (tests "unknown sorts/filters honestly").
29 const RALLIES: RallySegment[] = [
30   mk(1, 12, 8, 9),
31   mk(2, 41, 6, 5),
32   mk(3, 63, 21, 19),
33   mk(4, 95, 6 /* no shot_count */),
34   mk(5, 130, 12, 11),
35   mk(6, 158, 29, 24),
36 ];
37
38 describe("parseQuery ? duration filters", () => {
39   it("'over 10s' ? duration > 10", () => {
40     expect(parseQuery("over 10s").filters).toEqual([{ field: "duration", op: ">", value:
10 }]);
41   });
42   it("accepts varied units and phrasings", () => {
43     expect(parseQuery("longer than 10 seconds").filters[0]).toEqual({ field: "duration",
op: ">", value: 10 });
44     expect(parseQuery("under 5 sec").filters[0]).toEqual({ field: "duration", op: "<",
value: 5 });
45     expect(parseQuery("at least 8s").filters[0]).toEqual({ field: "duration", op: ">=",
value: 8 });
46     expect(parseQuery("at most 8 seconds").filters[0]).toEqual({ field: "duration", op:
"<=", value: 8 });
47     expect(parseQuery("exactly 7s").filters[0]).toEqual({ field: "duration", op: "=",
value: 7 });
48   });
49   it("converts minutes to seconds", () => {
50     expect(parseQuery("over 1 minute").filters[0]).toEqual({ field: "duration", op: ">",
value: 60 });
51     expect(parseQuery("at least 1.5m").filters[0]).toEqual({ field: "duration", op:
">=", value: 90 });
52   });
53   it("handles symbols and trailing comparators", () => {
54     expect(parseQuery("? 10s").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
55     expect(parseQuery("> 10").filters[0]).toEqual({ field: "duration", op: ">", value:
10 });
56     expect(parseQuery("10s+").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
57     expect(parseQuery("10s or longer").filters[0]).toEqual({ field: "duration", op:
">=", value: 10 });
58     expect(parseQuery("8s or less").filters[0]).toEqual({ field: "duration", op: "<=",
value: 8 });
59   });
60   it("does NOT consume 'no more than' as 'more than'", () => {
61     expect(parseQuery("no more than 5s").filters[0]).toEqual({ field: "duration", op:
"<=", value: 5 });
62   });
63   it("treats a bare 'Ns' (no comparator) as not a filter", () => {
64     expect(parseQuery("10s").filters).toEqual([]);
65     expect(parseQuery("10s").recognized).toBe(false);
66   });
67 });

```

```

68
69     describe("parseQuery ? shot_count filters (noun-keyed)", () => {
70         it("routes the 'shots' noun to shot_count, not duration", () => {
71             expect(parseQuery("over 10 shots").filters[0]).toEqual({ field: "shot_count", op:
72 ">", value: 10 });
73             expect(parseQuery("at least 8 shots").filters[0]).toEqual({ field: "shot_count", op:
74 ">=", value: 8 });
75             expect(parseQuery("fewer than 5 shots").filters[0]).toEqual({ field: "shot_count",
76 op: "<", value: 5 });
77             expect(parseQuery("10+ shots").filters[0]).toEqual({ field: "shot_count", op: ">=",
78 value: 10 });
79         });
80     });
81
82     describe("parseQuery ? sort", () => {
83         const cases: Array<[string, ReturnType<typeof parseQuery>["sort"]]> = [
84             ["longest first", { key: "duration", dir: "desc" }],
85             ["shortest", { key: "duration", dir: "asc" }],
86             ["newest first", { key: "start_time", dir: "desc" }],
87             ["oldest first", { key: "start_time", dir: "asc" }],
88             ["in order", { key: "start_time", dir: "asc" }],
89             ["most shots", { key: "shot_count", dir: "desc" }],
90             ["fewest shots", { key: "shot_count", dir: "asc" }],
91         ];
92         it.each(cases)("%s' ? %o", (input, expected) => {
93             expect(parseQuery(input).sort).toEqual(expected);
94         });
95     });
96
97     describe("parseQuery ? limit (and its implied sort)", () => {
98         it("'top 15' ? limit 15, sort longest-first", () => {
99             const q = parseQuery("top 15");
100             expect(q.limit).toBe(15);
101             expect(q.sort).toEqual({ key: "duration", dir: "desc" });
102         });
103         it("'first 10' ? limit 10, sort oldest-first", () => {
104             const q = parseQuery("first 10");
105             expect(q.limit).toBe(10);
106             expect(q.sort).toEqual({ key: "start_time", dir: "asc" });
107         });
108         it("'15 longest' / '5 shortest' carry both limit and sort", () => {
109             expect(parseQuery("15 longest")).toMatchObject({ limit: 15, sort: { key: "duration",
110 dir: "desc" } });
111             expect(parseQuery("5 shortest")).toMatchObject({ limit: 5, sort: { key: "duration",
112 dir: "asc" } });
113         });
114         it("an explicit sort wins over the limit's implied one ('top 15 longest')", () => {
115             const q = parseQuery("top 15 longest");
116             expect(q.limit).toBe(15);
117             expect(q.sort).toEqual({ key: "duration", dir: "desc" });
118         });
119         it("combines a filter with a limit ('over 10s top 5')", () => {
120             const q = parseQuery("over 10s top 5");
121             expect(q.filters).toEqual([ { field: "duration", op: ">", value: 10 } ]);
122             expect(q.limit).toBe(5);
123         });
124     });
125
126     describe("parseQuery ? unavailable concepts are flagged, never faked", () => {
127         it("flags shot-type words while still applying the available clause", () => {
128             const q = parseQuery("smashes over 10s");
129             expect(q.filters).toEqual([ { field: "duration", op: ">", value: 10 } ]); // honoured
130             expect(q.unavailable).toEqual([ { token: "smashes", kind: "shot-type" } ]); // flagged
131         });
132         it("flags player and score concepts", () => {

```

```

127     expect(parseQuery("opponent rallies").unavailable).toContainEqual({ token:
"opponent", kind: "player" });
128     expect(parseQuery("game point").unavailable).toContainEqual({ token: "game point",
kind: "score" });
129     });
130     it("does not confuse 'shots' (count) with a shot-type", () => {
131         expect(parseQuery("over 10 shots").unavailable).toEqual([]);
132     });
133     it("a pure unavailable query is not recognized as actionable", () => {
134         const q = parseQuery("rallies with drops");
135         expect(q.recognized).toBe(false);
136         expect(q.unavailable).toContainEqual({ token: "drops", kind: "shot-type" });
137     });
138 });
139
140 describe("parseQuery ? unrecognized / empty input", () => {
141     it("returns recognized=false with nothing to apply", () => {
142         for (const s of ["", " ", "hello there", "asdf"]) {
143             const q = parseQuery(s);
144             expect(q.recognized).toBe(false);
145             expect(q.filters).toEqual([]);
146             expect(q.sort).toBeUndefined();
147             expect(q.limit).toBeUndefined();
148         }
149     });
150 });
151
152 describe("sortRallies", () => {
153     it("sorts by duration desc / asc, stably", () => {
154         expect(sortRallies(RALLIES, { key: "duration", dir: "desc" })).map((r) =>
r.id)).toEqual([6, 3, 5, 1, 2, 4]);
155         expect(sortRallies(RALLIES, { key: "duration", dir: "asc" })).map((r) =>
r.id)).toEqual([2, 4, 1, 5, 3, 6]);
156     });
157     it("puts rallies with a missing field LAST regardless of direction", () => {
158         // rally 4 has no shot_count ? last whether asc or desc.
159         expect(sortRallies(RALLIES, { key: "shot_count", dir: "desc" })).map((r) =>
r.id)).toEqual([6, 3, 5, 1, 2, 4]);
160         expect(sortRallies(RALLIES, { key: "shot_count", dir: "asc" })).map((r) =>
r.id)).toEqual([2, 1, 5, 3, 6, 4]);
161     });
162     it("does not mutate the input array", () => {
163         const ids = RALLIES.map((r) => r.id);
164         sortRallies(RALLIES, { key: "duration", dir: "desc" });
165         expect(RALLIES.map((r) => r.id)).toEqual(ids);
166     });
167 });
168
169 describe("runQuery ? end to end", () => {
170     it("selects every rally over 10s, in chronological order (no sort)", () => {
171         const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("over 10s"));
172         expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]); // unsorted ? input
order
173         expect(selectedIds).toEqual([3, 5, 6]); // 21s, 12s, 29s
174     });
175     it("'top 3 longest' sorts all and selects the 3 longest", () => {
176         const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("top 3 longest"));
177         expect(ordered.map((r) => r.id)).toEqual([6, 3, 5, 1, 2, 4]); // all, longest-first
178         expect(selectedIds).toEqual([6, 3, 5]);
179     });
180     it("a shot filter excludes rallies that have no shot_count", () => {
181         const { selectedIds } = runQuery(RALLIES, parseQuery("at least 10 shots"));
182         expect(selectedIds).toEqual([3, 5, 6]); // rally 4 (no shot_count) excluded
183     });
184     it("filter + limit interact (longest 2 of the >10s rallies)", () => {

```

```

185     const { selectedIds } = runQuery(RALLIES, parseQuery("over 10s top 2"));
186     expect(selectedIds).toEqual([6, 3]); // sorted longest-first then capped at 2
187   });
188   it("an unrecognized query selects nothing and leaves order untouched", () => {
189     const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("smashes only"));
190     expect(selectedIds).toEqual([]);
191     expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]);
192   });
193 });
194
195 describe("labels", () => {
196   it("filterLabel renders honest field + symbol + unit", () => {
197     expect(filterLabel({ field: "duration", op: ">", value: 10 })).toBe("length > 10s");
198     expect(filterLabel({ field: "shot_count", op: ">=", value: 8 })).toBe("shots ? 8");
199   });
200   it("sortLabel reads naturally for every key/dir", () => {
201     expect(sortLabel({ key: "duration", dir: "desc" })).toBe("longest first");
202     expect(sortLabel({ key: "duration", dir: "asc" })).toBe("shortest first");
203     expect(sortLabel({ key: "start_time", dir: "desc" })).toBe("newest first");
204     expect(sortLabel({ key: "start_time", dir: "asc" })).toBe("oldest first");
205     expect(sortLabel({ key: "shot_count", dir: "desc" })).toBe("most shots first");
206     expect(sortLabel({ key: "shot_count", dir: "asc" })).toBe("fewest shots first");
207   });
208   it("summarize lists filter, sort and limit clauses", () => {
209     expect(summarize(parseQuery("over 10s top 5"))).toEqual(["length > 10s", "longest
first", "top 5"]);
210     expect(summarize(parseQuery("hello"))).toEqual([]);
211   });
212 });
213

```

5. user (2026-06-20T16:34:14.088Z)

```

1   # Interactive per-rally selection ? reel builder (RallyPicker)
2
3   > **Status:** `IN PROGRESS` ? design **APPROVED by owner 2026-06-20**; building the MVP
(P1?P5). Downstream milestones (multi-video auth, per-rally preview endpoint, any `/api/compile`
default change) remain owner-gated. . **Owner:** Avi . **Created:** 2026-06-20 . **Last
updated:** 2026-06-20
4   > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5   > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6   > **Relation:** second major tracked work item in the `khelsutra` repo (peer of
`REQUEST_ACCESS_FORM.md`); mirrors the engine repo's `PROJECT_DOC_TEMPLATE` discipline.
Primarily realizes alpha screen **A5 Highlights** (engine repo `UI_PROPOSAL.md:47`, exec-plan
**M4** `UI_ALPHA_EXECUTION_PLAN.md:39`); its correction-loop extension realizes **A4 Rally
Review** (`UI_PROPOSAL.md:46`, exec-plan **M3** `:38`). Consumes the engine's `/api/query` +
`/api/compile` contract.
7   > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9   ---
10
11  ## 0. TL;DR
12  Build the **pick-rallies ? reel** surface as a **hybrid**: a selectable rally
**list/grid** is the workhorse; a **deterministic query bar** ("rallies over 10s, longest
first") pre-selects into the **same** selection set; a sticky footer compiles the selection into
one downloadable MP4. The entire MVP runs on **existing engine endpoints** ? `POST /api/query` ?
`POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396,
307-328]` ? with **zero new engine intelligence and zero new endpoints**. The single most
important caveat: queries and chips are scoped to **length / count / order / shot-count only**;
```

final compiled reel can stream ? so MVP selects on text metadata, and visual preview/timeline is a deliberate later milestone.

13

14 **## 1. Problem & goal**

15 Today the alpha can detect rallies and stitch a reel, but reel-building is either a ****silent `stitching.min\_shot\_count` threshold**** ``[verified backend/main.py:362-383]`` or a static checkbox list whose data plumbing is even broken ? the bundled static UI reads ``data.rallies`` while the API returns ``play_segments``, so it renders nothing ``[verified backend/api/static/app.js:344 vs backend/main.py:340]``. The owner's framing is ****dynamic per-video selection, not a fixed threshold****: a coach should say "give me the long rallies from this session" and get exactly those, then fine-tune by hand.

16

17 ****The concrete outcome ("good")**** A coach uploads a 40-minute session; the engine finds 60 rallies. They type ****"top 15 longest"**, eyeball the auto-selected cards, untick two warm-up rallies, name it ****"Tuesday ? long points"**, and download a ~6-minute reel ? without learning any threshold knob.

18

19 ****Goal**** the smallest ***honest*** surface that turns engine-detected rallies into a user-curated, downloadable reel, with both ****fast**** (query) and ****precise**** (checkbox) selection sharing ****one**** state.

20

21 ****Non-goals (alpha)**** no conversational LLM (``UI_PROPOSAL.md:66`` defers it); no shot-type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt streaming; no multi-tenant auth (single-box alpha).

22

23 **## 2. Decisions locked**

24 | # | Decision | Source / date | Implication |

25 |---|-----|-----|-----|

26 | D1 | Dominant surface = selectable rally ****list/grid**** | Synthesis 2026-06-20 (Grid scored 15, fit-to-data 4) | Scales to long matches via bulk+filter; shows honest fields only |

27 | D2 | Fast entry = ****deterministic**** client-side ``parseQuery()`` (no LLM) | ``[verified UI_PROPOSAL.md:66]`` conversational QA deferred | Predictable, unit-testable, offline, zero API cost |

28 | D3 | ****One**** shared ``Set<segment_id>``, mutated by query + checkbox + (future) timeline | Synthesis 2026-06-20 | No second compile path can diverge from what the user sees |

29 | D4 | Compile via existing ``POST /api/compile {video_id, segment_ids, output_filename}`` | ``[verified backend/main.py:342-396]`` | Selection order is harmless ? sticher ``_merge_overlapping`` de-dupes ``[verified compiler.py:80-105, 311-317]`` |

30 | D5 | Query grammar (alpha) = length, count, order, ****shot_count only**** | ``[verified database.py:122-135, gemini.py:470-485]`` | Only fields the payload actually returns; grey out the rest |

31 | D6 | MVP backend = ****zero new endpoints**** | ``[verified]`` reuse map (\$4) | Ships the full loop on the already-shipped contract |

32 | D7 | Replace the silent ``min_shot_count`` cut with ****explicit user selection**** | Owner framing + ``[verified main.py:362-383]`` | UI ***warns*** about the floor; it never hides why a rally vanished |

33 | D8 | ****Default selection = ON**** (curate-by-removal) | Owner 2026-06-20 | Rallies load all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX assumes a non-empty start |

34 | D9 | ****Preview-before-select is NOT a ship-blocker**** for the MVP | Owner 2026-06-20 | MVP selects on text metadata + previews the ***final compiled reel***; the source/clip-stream endpoint (per-rally preview) is the next milestone (P9), not MVP |

35 | D10 | ****MVP is job-tethered**** (carry ``result.video_id``); no ``GET /api/videos`` | Owner 2026-06-20 | Smallest honest slice; multi-video home + identity scoping is P8 (gated) |

36 | D11 | ``min_shot_count``: ****warn**, do NOT override** in the MVP | Owner 2026-06-20 | The UI surfaces the floor; an explicit-selection override of the compile default is deferred (would trigger the Launch Report convention) |

37

38 **## 3. Foundation ? what the engine gives us today** ``[verified this session]``

39 The whole design rests on the engine's real, already-shipped contract. Read against the engine repo (``badminton-highlight-indexer``):

40

41 - ****The rally store is `play\_segments`**** ? columns ``id, video_id, start_time, end_time, duration, server, receiver, metadata`` ``[verified database.py:124-135]``. ``id`` is the integer the UI selects and passes to compile; ``start_time/end_time/duration`` are REAL seconds (duration

```

computed at insert).
42     - **Listing rallies = `POST /api/query {video_id}` ? `{play_segments:[...]}`** ? the
*only* rally-list path today; it JSON-parses `metadata` and joins `events` `[verified
main.py:331-340, database.py:427-475]`.
43     - **Building a reel = `POST /api/compile {video_id, segment_ids:[int],
output_filename}`** ? loads `videos.filepath`, filters all segments to the requested
`segment_ids`, applies the `min_shot_count` floor, extracts `time_pairs=[(start,end)]`, and
calls `VideoCompiler.compile_highlights(raw_path, segments, name)` `[verified main.py:342-396,
compiler.py:303]`, which **merges overlaps** then per-clip re-encodes ? concat ? one MP4
`[verified compiler.py:311-317]`. Selection order/duplicates are therefore safe.
44     - **Downloading = `GET /api/highlights/{video_id}/{filename}`** ? streams the compiled
MP4. Compile returns a **server-local filepath, NOT a URL** `[verified main.py:394]`, so the SPA
constructs this download URL itself. The compiled reel is the **only streamable video that
exists today** `[verified main.py:307-328]`.
45     - **The floor knob is readable** ? `GET /api/config` exposes `stitching.min_shot_count`
`[verified main.py:121-123]`, so the UI can warn before a selected rally is silently dropped.
46     - **Job entry** ? `GET /api/jobs/{job_id}` returns `result.video_id` `[verified
main.py:274-293]`, which is how the MVP reaches a video without a (nonexistent) list-videos
endpoint.
47
48     **Honest fields vs fabricated/roadmap (critical for the UI):**
49     - **Honest today** (default `default_segmenter: "gemini"`, per `config.json`
`[verified]`): `start_time`, `end_time`, `duration`, and `metadata.shot_count` (Gemini's
`shuttle_exchange_count`, fallback `max(3, int(dur/0.8))`) + `metadata.shot_count_confidence` (a
**string**, often `"Unknown"`) `[verified gemini.py:470-485]`. On the gemini path
`server/`receiver` are written `None` ("no fabricated data") `[verified gemini.py:480-487]`.
50     - **Fabricated ? do NOT surface:** the `motion` demo segmenter fills `server/`receiver`
via `random.sample(...)`, a random `shot_count`, and random `events` (serve/smash/foul/winner)
`[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not
ground truth and must stay hidden.
51     - **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point
score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified
database.py whole-schema 122-148]`.
52     - **Correction-loop columns exist but are dead:**
`candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes
them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6,
`UI_ALPHA_EXECUTION_PLAN.md`).
53
54     ## 4. Design / architecture
55
56     **Reuse map (existing ? `[verified]`, no change for MVP):** `POST /api/query` (list) ?
`POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET
`/api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the
SPA** in `khelsutra` `web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small
set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.
57
58     **4.1 Components (net-new SPA)**
59     - **QueryBar + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ?
`{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill**
("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-
correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM,
no network call.
60     - **RallyList/Grid** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss`
start, duration (`end?start`), and a shot-count + confidence chip **only when
`metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per
card; the thumbnail area is a placeholder (no endpoint yet).
61     - **`useRallySelection()` hook** ? owns `selectedIds:Set<number>`, `displayOrder`,
`apply(queryResult)`, `toggle(id)`, and bulk Select-all / Clear / Invert. Unit-tested (matches
the exec-plan's "API client + review-state logic" bar).
62     - **SelectionFooter (sticky)** ? live "N clips / total M:SS"; a **`min_shot_count`
warning** when a selected rally would be dropped at compile (read from `GET /api/config`); reel-
name input; one **Build** button.
63     - **CompileResult** ? `<video src=GET /api/highlights/...>` preview of the *compiled
reel* + a download link.
64     - **TypedApiClient** ? `query`, `compile`, `buildDownloadUrl`; the typed seam to the

```

```

engine.
65
66     **4.2 Data flow** `[verified anchors inline]`
67     Gemini segmenter (process time) writes `play_segments {id, start/end_time, duration,
metadata.shot_count/confidence/detector}` `[gemini.py:461-498]`
68     ? SPA `POST /api/query {video_id}` ? `{play_segments:[...]}` `[main.py:331-340]`
(**fix** the static UI's `data.rallies` read ? `play_segments` `[app.js:344]`)
69     ? normalize to `RallyCard[]` + one shared `selectedIds:Set`;
`server`/`receiver`/`events` are present in the payload but motion-only fabrications ? **not
surfaced** `[motion.py:191-198]`
70     ? all input modes mutate the **same** `selectedIds`: (a) `parseQuery()` predicate over
duration/ordinal/shot_count; (b) checkbox toggle + bulk ops; (c) *(post-MVP)* timeline band
71     ? footer recomputes count + ?duration **client-side**, and warns for any selected rally
with `shot_count < stitching.min_shot_count` `[main.py:362-383]`
72     ? **Build** ? `POST /api/compile {video_id, segment_ids: [...selectedIds],
output_filename}` ? stitcher de-dupes via `_merge_overlapping` `[compiler.py:311-317]`, cuts
`time_pairs` from `videos.filepath`, re-encodes per clip ? concat ? one MP4 `[main.py:386-393,
compiler.py:303]`
73     ? compile returns a **server-local filepath, not a URL** `[main.py:394]` ? SPA builds
`GET /api/highlights/{video_id}/{output_filename}` to preview in `` + download
`[main.py:307-328]`.
74
75     **4.3 Engine API additions (none for MVP; tagged against real gaps)**
76     | Endpoint | Purpose | Status |
77     |---|---|---|
78     | `POST /api/query` | List rallies (only path) ? fix SPA to read `play_segments` |
**exists** |
79     | `POST /api/compile` | Stitch selected `segment_ids` ? MP4 | **exists** |
80     | `GET /api/highlights/{video_id}/{filename}` | Stream/download compiled reel |
**exists** |
81     | `GET /api/config` | Read `stitching.min_shot_count` for the warning | **exists** |
82     | `GET /api/jobs/{job_id}` | Carry `result.video_id` into the picker | **exists** |
83     | `POST /api/compile` ? add `download_url` | Return `/api/highlights/...` so SPA stops
hand-building it (additive, low-risk; today returns only a filepath `main.py:394`) | **modify**
|
84     | `GET /api/health` | Engine-offline banner ping the SPA is specified to use
(`HOSTING_PLAN.md:42`) | **new** |
85     | `GET /api/videos/{video_id}/rallies` | RESTful/cacheable alias returning the same
`{play_segments}` shape | **new** |
86     | `GET /api/videos` | List a user's videos for a Match picker (only `get_video(id)`
exists, `database.py:418`) ? needs DB enumerate + identity scoping | **new** |
87     | `GET /api/videos/{video_id}/source` (Range) *or* `/clip?start=&end=` | Stream
source/per-span video so a rally previews before selection ? neither exists | **new** |
88     | `GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail` | Per-rally poster still
(no thumbnail column; WASB frames are transient) | **new** |
89     | `PATCH /api/segments/{id}` | Persist accept/reject/nudge ?
`candidate_segments.human_label/notes/...` (columns exist, unwritten ? corpus loop) | **new** |
90
91     **? Dead-ends / do-NOT:** never surface `server`/`receiver`/`events` (motion-only
randoms `[motion.py:191-198]`); never consume shot-type/player/score (no data); never render a
confidence **meter** ? `shot_count_confidence` is a Gemini *string* (often `"Unknown"`) and
`play_segments` has **no** numeric confidence column.
92
93     ## 5. Risk table
94     | Risk | Severity | Mitigation |
95     |---|---|---|
96     | **No source/clip/frame/thumbnail endpoint** ? only compiled reels stream
`[main.py:307-328]` | High (UX) | MVP selects on text metadata + previews the *final* reel;
preview/timeline deferred to M3 behind a new endpoint |
97     | `min_shot_count` silently drops selected rallies `[main.py:362-383]` | High (looks
broken) | Footer warns *before* Build; D7 makes selection explicit; an override is an owner gate
(Launch-Report-worthy `[memory: launch-report-convention]`) |
98     | No match-duration field ? a timeline must derive `total = max(end_time)`, mis-
rendering pre/post dead air | Med | Timeline deferred to M3; never over-promise it as a true
scrubber until source streams |

```

```

99      | `shot_count` is an *estimate* (`shuttle_exchange_count` w/ `max(3,int(dur/0.8))`
fallback) `[gemini.py:470-485]` | Med | Label "approx."; confidence shown as the raw string, not
a meter |
100     | No auth / user scoping anywhere (CORS `*`, `videos.user_id` never set/filtered) | High
for multi-tester | Fine for single-box local alpha; a ship-blocker for any shared host ? gate
`GET /api/videos` behind PR-3 scoping |
101     | compile returns a filepath not a URL `[main.py:394]` | Low | Client builds the URL for
MVP; `download_url` field (M1) removes the brittleness |
102     | Surface sprawl (query + grid + future timeline + future correction) | Med | Deliberate
IA: query+grid = one screen; correction = separate ? corpus screen (owner call, $8) |
103
104     ## 6. Honest limits ? what this does NOT do
105     A green SPA build + passing `parseQuery`/selection unit tests prove the **picker logic
only** ? **not** that the engine's rally boundaries are correct (the measured **rally-END**
weakness is a separate quality track, unaffected by this UI). "Conversational" here means a
**deterministic four-dimension parser**, not natural-language understanding. Nothing is
implemented in `web/` yet ? this is `[design]`. The feature curates *what the engine already
found*; it cannot surface a rally the detector missed, nor split/merge a mis-bounded one (that's
the M4 correction loop).
106
107     ## 7. Deliverables & progress tracker ? **source of truth**
108     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. CI (`npm test`) must stay green; merge to `master`
auto-deploys the site (docs/`web/` changes don't touch the live `site/` Worker).
109
110     | ID | Deliverable | Depends on | Gated? | Status | PR |
111     |---|-----|-----|-----|-----|---|
112     | P0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | No | ? |
[11](https://github.com/avidullu/khelsutra/pull/11) |
113     | P1 | SPA scaffold (Vite+React+TS+Tailwind) in `web/` + typed API client; dev proxy to
engine `:8000` | P0 | No | ? | [14](https://github.com/avidullu/khelsutra/pull/14) |
114     | P2 | RallyList/Grid (honest fields only) + `useRallySelection()` hook (unit-tested) |
P1 | No | ? | (this PR) |
115     | P3 | `parseQuery()` + parsed-pill + example chips (unit-tested; greys out unavailable
tokens) | P2 | No | ? | ? |
116     | P4 | SelectionFooter + `min_shot_count` warning (from `GET /api/config`) | P2 | No | ?
| ? |
117     | P5 | Build ? `POST /api/compile` ? client-built download URL + `` preview of
the reel | P2 | No | ? | ? |
118     | P6 | *(engine repo)* `GET /api/health` + offline banner; `/api/compile` returns
`download_url` | ? | No | ? | ? |
119     | P7 | *(engine repo)* `GET /api/videos/{id}/rallies` RESTful alias | ? | No | ? | ? |
120     | P8 | *(engine repo)* `GET /api/videos` + PR-3 identity scoping ? A1 Matches screen |
P7 | ? owner | ? | ? |
121     | P9 | *(engine repo)* source/clip stream + timeline + per-rally preview/thumbnail | P8
| ? owner | ? | ? |
122     | P10 | *(engine repo)* `PATCH /api/segments/{id}` corpus loop (PR-4/B6) ? A4 Rally
Review | P9 | ? owner | ? | ? |
123
124     **Milestone rollup:** **M0/MVP = P1?P5** (pick?reel on existing API, zero new engine
endpoints) · **M1 = P6?P7** (RESTful + reliability polish) · **M2 = P8** (multi-video home) ·
**M3 = P9** (per-rally preview/timeline) · **M4 = P10** (corpus correction loop) · **M5** = swap
`parseQuery()` for localized NL behind the same seam (gated on shot/player data landing).
125
126     ## 8. Open questions ? owner / external
127     **Blocking gates ? ? RESOLVED 2026-06-20 (owner) ? see D8?D11 in $2:**
128     1. ~~Default selection ON vs OFF?~~ ? **ON / curate-by-removal** (D8).
129     2. ~~Is preview-before-select a ship-blocker?~~ ? **No** ; text-metadata MVP, preview is
P9 (D9).
130     3. ~~Job-tethered vs multi-video MVP?~~ ? **Job-tethered** (D10).
131     4. ~~`min_shot_count` override?~~ ? **Warn, don't override** in MVP (D11).
132
133     **Still open (nice-to-knows, non-blocking):**
134     5. May user-facing copy call `parseQuery()` "conversational", given conversational QA is
explicitly deferred (`UI_PROPOSAL.md:66`)? Risk of over-promising NL. *Owner positioning/honesty

```

```

call.* *(Lean: avoid the word; show the parsed-pill instead.)*
135     6. One screen for query+grid (recommended) vs a separate ? corpus/correction screen for
A4? *Owner positioning call.*
136
137     ## 9. Definition of done
138     - [ ] From a finished job, a user can list rallies, query/select, and download a reel ?
on **existing** endpoints.
139     - [ ] Only honest fields (ordinal / duration / shot_count+confidence-when-present) are
shown; fabricated `server`/`receiver`/`events` are hidden.
140     - [ ] `parseQuery()` covers length/count/order/shot_count, greys out unavailable tokens,
shows a visible parsed-pill, and is unit-tested.
141     - [ ] Selection hook is unit-tested; the `min_shot_count` floor is surfaced, not hidden.
142     - [ ] `npm run build` (and `npm test`) clean; the static UI's `data.rallies` bug is gone
(or the static UI is superseded by the SPA).
143     - [ ] $7 tracker + this DoD updated; doc archived when P1?P5 land (MVP).
144
145     ## 10. References
146     **Engine code anchors `[verified this session]`:** `backend/main.py:121-123` (config),
`:274-293` (jobs), `:307-328` (highlights download), `:331-340` (query), `:342-396` (compile +
`min_shot_count` floor + filepath-not-URL); `backend/infrastructure/database.py:124-135`
(play_segments), `:166-168` (dead correction columns), `:418` (no list-videos), `:427-475`
(query read path + events join); `backend/pipeline/segmenters/gemini.py:461-498, 470-485`
(honest shot_count, server/receiver=None); `backend/pipeline/segmenters/motion.py:191-198`
(fabricated fields); `backend/pipeline/stitcher/compiler.py:80-105, 303, 311-317` (merge/de-dupe
+ entypoint); `backend/api/static/app.js:344` (`data.rallies` bug); `config.json`
(`default_segmenter: "gemini"`).
147     **Engine docs:** `UI_PROPOSAL.md:46-47, 66`; `UI_ALPHA_EXECUTION_PLAN.md:38-39`;
`HOSTING_PLAN.md:42`; `OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55` (conversational seam built now,
model punted). **This repo:** `web/README.md`, `REQUEST_ACCESS_FORM.md`.
148
149     ---
150     ### Changelog
151     - 2026-06-20 ? Initial `DRAFT`. Consolidated three candidate designs (Timeline / Grid /
RallyPicker) into the hybrid recommendation via a multi-agent design pass; every engine-fact
anchor re-verified against source before writing.
152     - 2026-06-20 ? Owner **approved** the recommendations; locked D8?D11 (default ON ·
preview not a ship-blocker · job-tethered MVP · warn-don't-override the floor); status `DRAFT` ?
IN PROGRESS`; P0 done.
153

```

6. assistant (2026-06-20T16:34:16.458Z)

Now let me read the remaining files for the React wiring analysis.

7. user (2026-06-20T16:34:17.415Z)

```

1      // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2      // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3      // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4      // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5      // greyed out as "not available yet" ? never silently faked.
6      import { useMemo, useState } from "react";
7      import type { FormEvent } from "react";
8      import { parseQuery, summarize } from "../lib/query";
9      import type { ParsedQuery } from "../lib/query";
10
11      interface Props {
12          /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection). */
13          onApply: (q: ParsedQuery) => void;
14      }

```

```

15
16 // Each example is a real, parseable query ? tapping one fills the box AND applies it.
17 const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "at least 8 shots",
18 "first 10"];
19
20 const CHIP = "rounded-full px-2.5 py-0.5 text-xs font-medium";
21
22 export function QueryBar({ onApply }: Props) {
23   const [value, setValue] = useState("");
24   const parsed = useMemo(() => parseQuery(value), [value]);
25   const clauses = summarize(parsed);
26   const typed = value.trim().length > 0;
27
28   const applyText = (text: string) => {
29     setValue(text);
30     onApply(parseQuery(text));
31   };
32   const submit = (e: FormEvent) => {
33     e.preventDefault();
34     if (parsed.recognized) onApply(parsed);
35   };
36
37   return (
38     <div>
39       <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
40         <input
41           type="text"
42           value={value}
43           onChange={(e) => setValue(e.target.value)}
44           placeholder="e.g. over 10s · top 15 longest · at least 8 shots"
45           aria-label="Filter rallies by length, count or order"
46           className="min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-white
47 px-4 py-2 text-sm text-ink outline-none focus:border-green"
48         />
49         <button
50           type="submit"
51           disabled={!parsed.recognized}
52           className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
53 disabled:opacity-40"
54         >
55           Apply
56         </button>
57       </form>
58
59       {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
60       */}
61
62       <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
63 items-center gap-1.5">
64         {clauses.map((c) => (
65           <span key={c} className={CHIP + " bg-green/15 text-green"}>
66             {c}
67           </span>
68         ))}
69         {parsed.unavailable.map((u) => (
70           <span
71             key={u.token}
72             title={`"${u.token}" needs ${u.kind} detection ? not available yet`}
73             className={CHIP + " bg-black/5 text-ink/45"}
74           >
75             <span className="line-through">{u.token}</span> · not available yet
76           </span>
77         ))}
78         {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
79           <span className="text-xs text-ink/50">Couldn't read that ? try ?over 10s? or
80 ?top 15 longest?</span>

```

```

74         })
75         {!typed && <span className="text-xs text-ink/50">Type a quick filter, or tap an
example.</span>}
76     </div>
77
78     <div className="mt-3 flex flex-wrap gap-2 text-sm">
79         <span className="text-ink/40">Try:</span>
80         {EXAMPLES.map((ex) => (
81             <button
82                 key={ex}
83                 type="button"
84                 onClick={() => applyText(ex)}
85                 className="rounded-full border border-black/15 px-3 py-1 text-xs
hover:border-green"
86             >
87                 {ex}
88             </button>
89         ))}
90     </div>
91 </div>
92 );
93 }
94

```

8. user (2026-06-20T16:34:17.890Z)

```

1     import { useMemo, useState } from "react";
2     import { useRallySelection } from "../hooks/useRallySelection";
3     import { RallyGrid } from "../components/RallyGrid";
4     import { QueryBar } from "../components/QueryBar";
5     import { mmss, pluralize } from "../lib/format";
6     import { runQuery, type ParsedQuery } from "../lib/query";
7     import { log } from "../lib/log";
8     import type { RallySegment } from "../api/types";
9
10    // Demo data until the picker is wired to the engine (POST /api/query) in the MVP flow
(P5).
11    // Shape matches the verified play_segments contract; only honest fields are shown.
12    const DEMO_RALLIES: RallySegment[] = [
13        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
14        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
15        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
16        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
17        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
18        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
19    ];
20
21    export default function App() {
22        const sel = useRallySelection(DEMO_RALLIES, "all");
23        // Display order is separate from selection: a query can re-sort the cards while the
shared
24        // selection set (sel) stays the single source of truth for what lands in the reel.
25        const [order, setOrder] = useState<RallySegment[]>(DEMO_RALLIES);
26
27        // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
28        const ordinalOf = useMemo(() => {
29            const m = new Map<number, number>();
30            [...DEMO_RALLIES].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));

```

```

31     return (id: number) => m.get(id) ?? 0;
32   }, []]);
33
34   const onApplyQuery = (q: ParsedQuery) => {
35     if (!q.recognized) {
36       log("info", "query.skip", { raw: q.raw, unavailable: q.unavailable.map((u) =>
u.token) });
37       return; // empty / unparseable ? leave the user's curation untouched
38     }
39     const { ordered, selectedIds } = runQuery(DEMO_RALLIES, q);
40     setOrder(ordered);
41     sel.apply(selectedIds);
42     log("info", "query.apply", {
43       filters: q.filters.length,
44       sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
45       limit: q.limit ?? null,
46       selected: selectedIds.length,
47     });
48   };
49
50   return (
51     <main className="min-h-screen bg-paper text-ink pb-24">
52       <header className="bg-ink text-white">
53         <div className="mx-auto flex max-w-5xl items-center gap-1 px-6 py-4">
54           <span className="text-xl font-extrabold tracking-tight">
55             Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
56           </span>
57         </div>
58       </header>
59
60       <section className="mx-auto max-w-5xl px-6 py-10">
61         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
62           per-rally selection · P3
63         </span>
64         <h1 className="mt-4 text-3xl font-extrabold tracking-tight">Pick the rallies for
your reel</h1>
65         <p className="mt-2 max-w-2xl text-ink/70">
66           Type a quick filter like <em>?top 15 longest?</em>, then fine-tune by tapping
cards. Everything starts{" "}
67           <strong>selected</strong> ? just drop the ones you don't want.{" "}
68           <span className="text-ink/50">(Demo data; the live list comes from the
engine.)</span>
69         </p>
70
71         <div className="mt-6">
72           <QueryBar onApply={onApplyQuery} />
73         </div>
74
75         <div className="mt-6 flex flex-wrap gap-2 text-sm">
76           <button type="button" onClick={sel.selectAll} className="rounded-full border
border-black/15 px-3 py-1 hover:border-green">Select all</button>
77           <button type="button" onClick={sel.clearAll} className="rounded-full border
border-black/15 px-3 py-1 hover:border-green">Clear</button>
78           <button type="button" onClick={sel.invert} className="rounded-full border
border-black/15 px-3 py-1 hover:border-green">Invert</button>
79         </div>
80
81         <div className="mt-5">
82           <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={sel.toggle}
ordinalOf={ordinalOf} />
83         </div>
84       </section>
85

```

```

86         <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
87             <div className="mx-auto flex max-w-5xl items-center gap-4 px-6 py-3">
88                 <span className="text-sm text-ink/80" data-testid="selection-summary">
89                     <strong className="text-ink">{pluralize(sel.count, "rally",
"rallies")}</strong> selected · {mmss(sel.totalSec)} total
90                 </span>
91                 <button
92                     type="button"
93                     disabled={sel.count === 0}
94                     title="Reel building wires up in P5"
95                     className="ml-auto rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
96                 >
97                     Build reel ({sel.count})
98                 </button>
99             </div>
100         </footer>
101     </main>
102 );
103 }
104

```

9. user (2026-06-20T16:34:18.368Z)

```

1      // React state wrapper over the pure selection helpers in lib/selection.ts.
2      // Default = ALL selected (D8: curate-by-removal) ? the user removes warm-ups.
3      import { useCallback, useMemo, useState } from "react";
4      import type { RallySegment } from "../api/types";
5      import * as sel from "../lib/selection";
6
7      export function useRallySelection(rallies: RallySegment[], initial: "all" | "none" =
"all") {
8          const [selected, setSelected] = useState<Set<number>>(() =>
9              initial === "none" ? sel.clear() : sel.selectAll(rallies),
10          );
11
12          const toggle = useCallback((id: number) => setSelected((s) => sel.toggle(s, id)), []);
13          const selectAll = useCallback(() => setSelected(sel.selectAll(rallies)), [rallies]);
14          const clearAll = useCallback(() => setSelected(sel.clear()), []);
15          const invert = useCallback(() => setSelected((s) => sel.invert(s, rallies)),
[rallies]);
16          const isSelected = useCallback((id: number) => selected.has(id), [selected]);
17          /** Replace the whole selection (the query bar writes the shared set here ? D3). */
18          const apply = useCallback((ids: Iterable<number>) => setSelected(new Set(ids)), []);
19
20          const totalSec = useMemo(() => sel.selectedDurationSec(rallies, selected), [rallies,
selected]);
21
22          return { selected, count: selected.size, totalSec, isSelected, toggle, selectAll,
clearAll, invert, apply };
23      }
24

```

10. user (2026-06-20T16:34:19.541Z)

```

1      // The selectable rally grid ? one card per play_segment, HONEST fields only:
2      // ordinal (by start order), mm:ss start, duration, and a shot-count chip ONLY when
present.
3      // Never renders server/receiver/events (motion-only fabrications,
PER_RALLY_SELECTION.md §4).
4      import type { RallySegment } from "../api/types";
5      import { mmss } from "../lib/format";
6
7      interface Props {

```

```

8     rallies: RallySegment[];
9     isSelected: (id: number) => boolean;
10    onToggle: (id: number) => void;
11    /** Stable chronological label for a rally (by start_time), so re-sorting via a query
12     * doesn't renumber the cards. Falls back to display position when absent. */
13    ordinalOf?: (id: number) => number;
14  }
15
16  export function RallyGrid({ rallies, isSelected, onToggle, ordinalOf }: Props) {
17    if (rallies.length === 0) {
18      return <p className="text-ink/60 text-sm">No rallies to show yet.</p>;
19    }
20    return (
21      <ul className="grid grid-cols-[repeat(auto-fill,minmax(150px,1fr))] gap-3 list-none
p-0 m-0">
22        {rallies.map((r, i) => {
23          const selected = isSelected(r.id);
24          const n = ordinalOf ? ordinalOf(r.id) : i + 1;
25          const shot = r.metadata?.shot_count;
26          const conf = r.metadata?.shot_count_confidence;
27          return (
28            <li key={r.id}>
29              <button
30                type="button"
31                onClick={() => onToggle(r.id)}
32                aria-pressed={selected}
33                aria-label={`Rally ${n}${selected ? ", in reel" : ", not in reel"}`}
34                className={
35                  "w-full text-left rounded-xl border p-3 transition " +
36                  (selected
37                    ? "border-green bg-green/10"
38                    : "border-black/10 bg-white hover:border-green/60")
39                )
40              >
41                <div className="flex items-center justify-between">
42                  <span className="font-semibold text-ink">Rally {n}</span>
43                  <span
44                    aria-hidden="true"
45                    className={
46                      "inline-flex h-5 w-5 items-center justify-center rounded-full text-
xs " +
47                      (selected ? "bg-green text-[#04140D]" : "border border-black/20
text-transparent")
48                    }
49                  >
50                    ?
51                  </span>
52                </div>
53                <div className="mt-1 text-sm text-ink/70">
54                  {mmss(r.start_time)} · {mmss(r.duration)}
55                </div>
56                {typeof shot === "number" && (
57                  <div className="mt-2 inline-block rounded-full bg-paper px-2 py-0.5
text-xs text-ink/70">
58                    ~{shot} shots{conf && conf !== "Unknown" ? ` · ${conf}` : ""}
59                  </div>
60                )}
61              </button>
62            </li>
63          );
64        })}
65      </ul>
66    );
67  }
68

```

11. user (2026-06-20T16:34:19.620Z)

```
1 // Types mirror the engine's verified contract (badminton-highlight-indexer).
2 // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3 // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5 export interface RallyMetadata {
6   /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7   shot_count?: number;
8   /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9   shot_count_confidence?: string;
10  detector?: string;
11  [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15   /** The integer the UI selects and passes to /api/compile as segment_ids. */
16   id: number;
17   start_time: number;
18   end_time: number;
19   duration: number;
20   /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21   server: string | null;
22   /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23   receiver: string | null;
24   metadata: RallyMetadata;
25   /** motion-only fabricated sub-events ? DO NOT surface. */
26   events?: unknown[];
27 }
28
29 export interface QueryResponse {
30   play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34   status: string;
35   /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36   filepath: string;
37 }
38
39 export interface EngineConfig {
40   stitching?: {
41     /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
42     min_shot_count?: number;
43     [key: string]: unknown;
44   };
45   [key: string]: unknown;
46 }
47
```

12. assistant (2026-06-20T16:34:26.543Z)

Now let me focus on the regex hygiene and edge cases. Let me trace through several inputs carefully. First, let me check the ``\b`` behavior with the ``s`` unit and the global regex state.

Let me verify my findings empirically by running the parser with Node.

13. user (2026-06-20T16:34:38.743Z)

Exit code 1

```
file:///C:/Users/avidu/AppData/Local/Temp/probe.mjs:17
  { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
```

```
(?:more|longer|over|above|greater))$${POST_NOUN}` , "g") },
    ^
```

SyntaxError: Invalid regular expression:

```
/(d+(?:.d+)?)(?:s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)■)?s*(?:+|or
(?:more|longer|over|above|greater))(?:s*(shots?)■)?/g: Nothing to repeat
    at new RegExp (<anonymous>)
    at file:///C:/Users/avidu/AppData/Local/Temp/probe.mjs:17:19
    at ModuleJob.run (node:internal/modules/esm/module_job:439:25)
    at async node:internal/modules/esm/loader:636:26
    at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:101:5)
```

Node.js v26.1.0

14. assistant (2026-06-20T16:34:44.408Z)

The heredoc ate the backslashes. Let me write the probe file directly with the Write tool to preserve escaping.

15. assistant (2026-06-20T16:35:02.759Z)

```
// Replicate the EXACT regex fragments from query.ts to probe behavior.
const NUM = "(\\d+(?:\\.\\d+)?)";
const UNIT = "(?:\\s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)\\b)?"
const POST_NOUN = "(?:\\s*(shots?)\\b)?"

function lead(alt, op) {
    return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}` , "g"), numAt: 1, unitAt: 2 };
}
const LEADING = [
    lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
    lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),
    lead("exactly|equal to|equals", "="),
    lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
    lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
];
const TRAILING = [
    { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater))$${POST_NOUN}` , "g") },
    { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
(?:less|fewer|shorter|under|below)$${POST_NOUN}` , "g") },
];
function toSeconds(n, unit) { return unit && /^m/.test(unit) ? n * 60 : n; }
function makeFilter(numStr, unit, op) {
    const n = Number(numStr);
    const isShots = !!unit && /^shots?$/ .test(unit);
    return isShots ? { field: "shot_count", op, value: n } : { field: "duration", op, value:
toSeconds(n, unit) };
}

const SORT_PATTERNS = [
    [/\b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, { key:
"duration", dir: "desc" }],
    [/\b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir: "asc" }],
    [/\b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time", dir: "desc"
}],
    [/\b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key: "start_time",
dir: "asc" }],
    [/\b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count desc)\b/, {
key: "shot_count", dir: "desc" }],
    [/\b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
"shot_count", dir: "asc" }],
];

function parse(raw) {
```

```
const normalized = raw.toLowerCase().replace(/?/g, ">=").replace(/?/g, "<=").replace(/,/g, "
").replace(/\s+/g, " ").trim();
const filters = [];
let working = normalized;
for (const pat of LEADING) {
    working = working.replace(pat.re, (full, ...g) => {
        filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
        return " ".repeat(full.length);
    });
}
for (const pat of TRAILING) {
    working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
        filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
        return " ".repeat(full.length);
    });
}
let sort;
for (const [re, s] of SORT_PATTERNS) {
    if (re.test(working)) { sort = s; break; }
}
let limit;
let m;
if ((m = working.match(/\b(\d+)\s+(longest|shortest)\b/)) || (m =
working.match(/\b(longest|shortest)\s+(\d+)\b/))) {
    const num = /\^d/.test(m[1]) ? m[1] : m[2];
    const word = /\^d/.test(m[1]) ? m[2] : m[1];
    limit = parseInt(num, 10);
    if (!sort) sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
} else if ((m = working.match(/\b(?:top|best)\s+(\d+)\b/))) {
    limit = parseInt(m[1], 10);
    if (!sort) sort = { key: "duration", dir: "desc" };
} else if ((m = working.match(/\b(?:first|earliest)\s+(\d+)\b/))) {
    limit = parseInt(m[1], 10);
    if (!sort) sort = { key: "start_time", dir: "asc" };
} else if ((m = working.match(/\b(?:last|latest|newest|most recent)\s+(\d+)\b/))) {
    limit = parseInt(m[1], 10);
    if (!sort) sort = { key: "start_time", dir: "desc" };
}
return { filters, working, sort, limit };
}

const tests = [
    "over 10.5.5s",
    "over 100000000000000000000s",
    "min 5s",
    "over 0 shots",
    "at least 8 shots over 10s",
    "more than 5 minutes long",
    "over -5s",
    "over 5s and over 8s",
    "10.s",
    "over 1,000 shots",
    "max 5 min",
    "over 10m",
    "min 5",
    "minimum 5s",
    "longest 0",
    "top 0",
    "top 9999999999999999999999",
    "over 5 minutes",
    "smaller than 8 mins",
];
for (const t of tests) {
    const r = parse(t);
    console.log(JSON.stringify(t), "=> filters:", JSON.stringify(r.filters), "sort:",
```

```
JSON.stringify(r.sort), "limit:", r.limit, "| working:", JSON.stringify(r.working));
}
```

16. user (2026-06-20T16:35:02.770Z)

File created successfully at: C:/Users/avidu/Projects/khelsutra/web/tmp_probe.mjs (file state is current in your context ? no need to Read it back)

17. user (2026-06-20T16:35:09.651Z)

[illegible]

18. assistant (2026-06-20T16:35:16.011Z)

Several confirmed issues. Let me focus on the most defensible ones and verify the key ones:

1. `**`over 1,000 shots`**` ? comma stripped to space, then ``over 1`` parses as ``duration > 1``, dropping the rest. Wrong: should be 1000 shots. (confirmed)
2. `**`over 10.5.5s`**` ? parses ``duration > 10.5``, leaving ``.5s`` garbage. (confirmed)
3. `**`top 0` / `longest 0`**` ? ``limit: 0`` which means ``slice(0,0)`` ? empty selection. Let me check the runQuery behavior.
4. `**Huge number**` ? ``1e21`` for limit, harmless but ``slice`` accepts it.

Let me now verify the global regex ``lastIndex`` state bleed across ``parseQuery`` calls, since ``test()`` on a global regex is stateful and ``SORT_PATTERNS`` are module-level.

19. assistant (2026-06-20T16:35:29.592Z)

```
// Probe global-regex lastIndex state bleed across calls.
// SORT_PATTERNS use NON-global regexes (no /g) ? test() is safe. But UNAVAILABLE_PATTERNS
// are GLOBAL (/g) and used with matchAll (which is safe per-call, fresh iterator).
// The LEADING/TRAILING regexes are GLOBAL and used via .replace() ? replace resets lastIndex.
// Let's confirm there is NO bleed by running the same input twice and a tricky sequence.
```

```
const SORT_PATTERNS = [
  [/\b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, { key:
"duration", dir: "desc" }],
];
```

```
// Simulate: is SORT_PATTERNS test() stateful? These are non-global so NO.
const re = SORT_PATTERNS[0][0];
console.log("sort re global?", re.global); // false ? safe
```

```
// Now the UNAVAILABLE patterns (global) used with matchAll in a loop:
```

```
const UNAVAILABLE = [
  ["shot-type", /\b(smash(?:es)?|clears?|drops?)\b/g],
];
function unavail(normalized) {
  const out = [];
  const seen = new Set();
  for (const [kind, r] of UNAVAILABLE) {
    for (const mm of normalized.matchAll(r)) {
      const token = mm[1];
      if (!seen.has(token)) { seen.add(token); out.push({ token, kind }); }
    }
  }
  return out;
}
```

```
// matchAll on a global regex: does it leave lastIndex dirty for the NEXT call?
console.log("call1", JSON.stringify(unavail("smashes and drops")));
console.log("lastIndex after matchAll:", UNAVAILABLE[0][1].lastIndex);
console.log("call2", JSON.stringify(unavail("smashes and drops")));
console.log("call3 (clears)", JSON.stringify(unavail("clears")));
```

```
// Now test the ReDoS surface: deeply alternation-nested NUM/UNIT with long input.
```

```
const NUM = "(\\d+(?:\\.\\d+)?)";
const UNIT = "(?:\\s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)\\b)?";
const POST_NOUN = "(?:\\s*(shots?)\\b)?";
const trailRe = new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater))${POST_NOUN}`, "g");
const big = "9".repeat(50000) + "s or more " ;
const t0 = Date.now();
trailRe.lastIndex = 0;
const matched = trailRe.test(big);
console.log("ReDoS trail test on 50k digits:", Date.now() - t0, "ms matched:", matched);
```

```
const leadRe = new RegExp(`(?:over)\\s*${NUM}${UNIT}`, "g");
const big2 = "over " + "9".repeat(200000);
const t1 = Date.now();
leadRe.lastIndex = 0;
console.log("lead test on 200k digits:", JSON.stringify(big2.replace(leadRe, "X")).length,
"took", Date.now()-t1, "ms");
```

20. user (2026-06-20T16:35:29.606Z)

File created successfully at: C:/Users/avidu/Projects/khelsutra/web/tmp_probe2.mjs (file state is current in your context ? no need to Read it back)

21. user (2026-06-20T16:35:35.563Z)

```
sort re global? false
call1 [{"token":"smashes","kind":"shot-type"}, {"token":"drops","kind":"shot-type"}]
lastIndex after matchAll: 0
```

```
call2 [{"token":"smashes","kind":"shot-type"}, {"token":"drops","kind":"shot-type"}]
call3 (clears) [{"token":"clears","kind":"shot-type"}]
ReDoS trail test on 50k digits: 0 ms matched: true
lead test on 200k digits: 3 took 0 ms
```

22. assistant (2026-06-20T16:35:39.568Z)

Good - so `matchAll` correctly does not leave `lastIndex` dirty (it operates on a clone), and the regexes are linear (no catastrophic backtracking). So ****no ReDoS and no state bleed**** ? those probes come back clean. That's a respected negative result.

Now let me verify the `limit: 0` end-to-end behavior in `runQuery`, and check the negative number and the `top 0` selection-wipe issue.

23. assistant (2026-06-20T16:35:47.174Z)

```
// Verify runQuery behavior with limit:0 and the parseInt overflow ? recognized semantics.
// Replicate runQuery + parse minimal.
const RALLIES = [
  { id: 1, start_time: 12, duration: 8, metadata: { shot_count: 9 } },
  { id: 2, start_time: 41, duration: 6, metadata: { shot_count: 5 } },
  { id: 3, start_time: 63, duration: 21, metadata: { shot_count: 19 } },
];
function sortRallies(rallies, sort) {
  return rallies.map((r,i)=>({r,i})).sort((a,b)=>{
    const av = sort.key==="duration"?a.r.duration:a.r.start_time;
    const bv = sort.key==="duration"?b.r.duration:b.r.start_time;
    if (av!==bv) return sort.dir==="asc"?av-bv:bv-av;
    return a.i-b.i;
  }).map(x=>x.r);
}
function runQuery(rallies, q) {
  if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
  const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
  let matched = ordered.filter(() => true);
  if (q.limit !== null) matched = matched.slice(0, q.limit);
  return { ordered, selectedIds: matched.map(r=>r.id) };
}

// "top 0": recognized true (limit !== null), sort set, limit 0 => slice(0,0) => []
const qTop0 = { recognized: true, sort: { key:"duration", dir:"desc" }, filters: [], limit: 0 };
console.log("top 0 =>", JSON.stringify(runQuery(RALLIES, qTop0)));

// huge limit
const qBig = { recognized: true, sort: { key:"duration", dir:"desc" }, filters: [], limit: 1e21 };
console.log("top 1e21 =>", JSON.stringify(runQuery(RALLIES, qBig).selectedIds));

// "longest 0" same effect -> wipes selection to empty, but RECOGNIZED is true so App applies it.
```

24. user (2026-06-20T16:35:47.191Z)

File created successfully at: C:/Users/avidu/Projects/khelsutra/web/tmp_probe3.mjs (file state is current in your context ? no need to Read it back)

25. user (2026-06-20T16:35:50.765Z)

```
top 0 => {"ordered":[{"id":3,"start_time":63,"duration":21,"metadata":{"shot_count":19}},{"id":1,"start_time":12,"duration":8,"metadata":{"shot_count":9}},{"id":2,"start_time":41,"duration":6,"metadata":{"shot_count":5}}],"selectedIds":[]}
top 1e21 => [3,1,2]
```

26. assistant (2026-06-20T16:36:03.530Z)

